

Mini Pinnacle

Complete Developer Guide

Full-Stack Enterprise Application — Step by Step

Next.js · TypeScript · Prisma · PostgreSQL · Tailwind CSS

Table of Contents

- [Phase 0 — Architecture Overview](#)
- [Phase 1 — Environment Setup](#)
- [Phase 2 — Project Structure](#)
- [Phase 3 — Global Layout — Sidebar & Header](#)
- [Phase 4 — TypeScript Basics](#)
- [Phase 5 — Database Design](#)
- [Phase 6 — Prisma Setup & Schema](#)
- [Phase 7 — Authentication \(NextAuth\)](#)
- [Phase 8 — Role-Based Authorization](#)
- [Phase 9 — Dashboard — KPI Cards & Filters](#)
- [Phase 10 — Projects / Accounts Listing](#)
- [Phase 11 — Create Project Form](#)
- [Phase 12 — Project Detail Page & Tabs](#)
- [Phase 13 — Service Lines Module](#)
- [Phase 14 — Opportunities Module](#)
- [Phase 15 — Users Module](#)
- [Phase 16 — Profile Page](#)
- [Phase 17 — API Architecture \(All Endpoints\)](#)
- [Phase 18 — Component Architecture](#)
- [Phase 19 — Custom Hooks](#)
- [Phase 20 — Error Handling & Loading States](#)
- [Phase 21 — Search & Filtering](#)
- [Phase 22 — Git Workflow](#)
- [Phase 23 — Testing Checklist](#)
- [Phase 24 — Final Project Tree](#)

Phase 0 — Architecture Overview

Before writing a single line of code, understand the full system we are building.

What is Mini Pinnacle?

Mini Pinnacle is an enterprise account-management dashboard. CXO and management users log in to see all their company accounts (clients), the services delivered to those accounts, opportunities being pursued, and user management — all in one place.

Request Flow

Every action you take in the browser travels through this chain:

```
Browser (user clicks something)
↓
Next.js (serves the React page, handles routing)
↓
React Components (renders the UI you see)
↓
API Route Handlers (app/api/... — the 'backend')
↓
Prisma ORM (translates TypeScript into SQL)
↓
PostgreSQL (permanent data storage)
```

Where Each Concern Lives

Concern	Location	Why There
Authentication	app/api/auth/ + middleware.ts	Checks identity on every request
Authorization	API route handlers	Backend-enforced — cannot be bypassed from browser
Business Logic	app/api/	Rules like 'only Manager+ can create projects'
Reusable UI	components/	Prevents copy-pasting the same code into every page
Database access	lib/prisma.ts (Prisma client)	Single shared connection to PostgreSQL
TypeScript types	types/	Shared type contracts between frontend and backend

Phase 1 — Environment Setup

Install Node.js

Download LTS version from <https://nodejs.org>. Run installer with default options.

Verify Installation

```
node -v      # should print v20.x.x or higher
npm -v       # should print 10.x.x or higher
```

Create the Next.js Project

```
cd Desktop
npx create-next-app@latest mini-pinnacle
```

Answer the prompts exactly like this:

Question	Answer	Why
TypeScript?	Yes	Type safety — prevents entire classes of bugs
ESLint?	Yes	Catches mistakes as you type
Tailwind CSS?	Yes	Our styling approach
src/ directory?	No	Keeps app/ at project root — simpler
App Router?	Yes	Modern Next.js routing — used throughout this guide
Customize import alias?	No	Default @/ alias is fine

Start the Project

```
cd mini-pinnacle
code .          # opens VS Code
npm run dev     # starts dev server
```

Open <http://localhost:3000> — you should see the default Next.js page.

Initialize Git & Push to GitHub

```
git init
git add .
git commit -m "chore: initial Next.js setup"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/mini-pinnacle.git
git push -u origin main
```

Install All Project Dependencies Now

Run these once at the start. We explain each as we use them.

```
npm install @prisma/client prisma
npm install next-auth@beta @auth/prisma-adapter
npm install bcryptjs
npm install @types/bcryptjs --save-dev
npm install zod
```

Phase 2 — Project Structure

Create this folder structure. Every folder has a specific purpose.

```
mini-pinnacle/
├── app/
│   ├── (auth)/
│   │   └── login/
│   │       └── page.tsx
│   ├── dashboard/
│   │   └── page.tsx
│   ├── projects/
│   │   ├── page.tsx
│   │   ├── new/
│   │   │   └── page.tsx
│   │   └── [id]/
│   │       └── page.tsx
│   ├── users/
│   │   └── page.tsx
│   ├── profile/
│   │   └── page.tsx
│   ├── api/
│   │   ├── auth/
│   │   │   └── [...nextauth]/
│   │   │       └── route.ts
│   │   ├── projects/
│   │   │   ├── route.ts
│   │   │   └── [id]/
│   │   │       └── route.ts
│   │   ├── users/
│   │   │   └── route.ts
│   │   └── service-lines/
│   │       └── route.ts
│   ├── layout.tsx
│   └── page.tsx
├── components/
│   ├── layout/
│   │   ├── Sidebar.tsx
│   │   └── Header.tsx
│   ├── dashboard/
│   │   ├── KpiCard.tsx
│   │   └── FilterBar.tsx
│   ├── projects/
│   │   ├── ProjectTable.tsx
│   │   ├── ProjectForm.tsx
│   │   ├── ProjectHeader.tsx
│   │   └── ProjectTabs.tsx
│   ├── users/
│   │   └── UserTable.tsx
│   └── common/
│       ├── Button.tsx
│       ├── Input.tsx
│       ├── Modal.tsx
│       └── LoadingSpinner.tsx
├── lib/
│   ├── prisma.ts
│   ├── auth.ts
│   └── permissions.ts
└── hooks/
    └── useProjects.ts
```

```

└─ useDebounce.ts
└─ types/
    └─ index.ts
└─ prisma/
    └─ schema.prisma
└─ middleware.ts

```

Folder Explanations

Folder	What Goes Here
app/	Pages and API routes. Folder name = URL path.
app/(auth)/	Route group — the parentheses mean (auth) does NOT appear in the URL. Used to apply a different layout to login page.
app/api/	Backend API endpoints. Files here are never shown in the browser — only called by fetch().
components/	Reusable UI pieces imported by pages.
lib/	Shared utility code: Prisma client, auth config, permission helpers.
hooks/	Custom React hooks — reusable state/fetch logic.
types/	TypeScript interfaces used throughout the app.
prisma/	Database schema definition file.
middleware.ts	Runs on every request — used to protect routes.

Phase 3 — Global Layout (Sidebar & Header)

File: components/layout/Sidebar.tsx

```
"use client";
import Link from "next/link";
import { usePathname } from "next/navigation";

const navItems = [
  { label: "Dashboard", href: "/dashboard", icon: "📊" },
  { label: "Projects", href: "/projects", icon: "📁" },
  { label: "Users", href: "/users", icon: "👥" },
  { label: "Profile", href: "/profile", icon: "👤" },
];

export default function Sidebar() {
  const pathname = usePathname();

  return (
    <aside className="w-64 h-screen bg-slate-900 text-white flex flex-col fixed left-0 top-0 z-40">
      <div className="px-6 py-5 text-xl font-bold border-b border-slate-700">
        🏠 Mini Pinnacle
      </div>
      <nav className="flex-1 px-3 py-4 space-y-1">
        {navItems.map((item) => {
          const isActive = pathname.startsWith(item.href);
          return (
            <Link
              key={item.href}
              href={item.href}
              className={`block px-3 py-2 rounded-md text-sm font-medium transition-colors ${
                isActive
                  ? "bg-blue-600 text-white"
                  : "text-slate-300 hover:bg-slate-800 hover:text-white"
              }`}
            >
              {item.icon} {item.label}
            </Link>
          );
        })}
      </nav>
    </aside>
  );
}
```

File: components/layout/Header.tsx

```
"use client";
import { signOut, useSession } from "next-auth/react";

export default function Header() {
  const { data: session } = useSession();

  return (
```

```

    <header className="h-16 bg-white border-b border-slate-200 flex items-center
justify-between px-6 sticky top-0 z-30">
      <h1 className="text-lg font-semibold text-slate-800">
        Welcome back, {session?.user?.name ?? "User"}
      </h1>
      <div className="flex items-center gap-4">
        <span className="text-xs bg-blue-100 text-blue-700 px-2 py-1 rounded-full
font-medium">
          {session?.user?.role ?? "User"}
        </span>
        <button
          onClick={() => signOut({ callbackUrl: "/login" })}
          className="text-sm text-slate-500 hover:text-red-500 transition-colors"
        >
          Logout
        </button>
      </div>
    </header>
  );
}

```

File: app/layout.tsx

```

import type { Metadata } from "next";
import "@/globals.css";
import { SessionProvider } from "next-auth/react";
import Sidebar from "@/components/layout/Sidebar";
import Header from "@/components/layout/Header";

export const metadata: Metadata = {
  title: "Mini Pinnacle",
  description: "Enterprise Account Management",
};

export default function RootLayout({
  children,
}): {
  children: React.ReactNode;
} {
  return (
    <html lang="en">
      <body className="antialiased bg-slate-50">
        <SessionProvider>
          <div className="flex">
            <Sidebar />
            <div className="ml-64 flex-1 flex flex-col min-h-screen">
              <Header />
              <main className="p-6 flex-1">{children}</main>
            </div>
          </div>
        </SessionProvider>
      </body>
    </html>
  );
}

```

Phase 4 — TypeScript Types

TypeScript adds types to JavaScript, catching errors before your code runs. Here are all the types used in Mini Pinnacle.

[File: types/index.ts](#)

```
// — Roles —————
export type Role = "SUPER_ADMIN" | "MANAGER" | "END_USER";

// — User —————
export interface User {
  id: string;
  name: string;
  email: string;
  role: Role;
  createdAt: string;
}

// — Project / Account —————
export interface Project {
  id: string;
  accountName: string;
  accountId: string;
  description?: string;
  ibu: string;
  region: string;
  industry: string;
  zone: string;
  status: "ACTIVE" | "INACTIVE" | "PENDING";
  priority: "HIGH" | "MEDIUM" | "LOW";
  tier: string;
  createdAt: string;
  updatedAt: string;
  servicelines?: ServiceLine[];
  opportunities?: Opportunity[];
}

// — Service Line —————
export interface ServiceLine {
  id: string;
  name: string;
  description?: string;
  projectId: string;
}

// — Opportunity —————
export interface Opportunity {
  id: string;
  name: string;
  description?: string;
  stage: string;
  priority: "HIGH" | "MEDIUM" | "LOW";
  estimatedValue?: number;
  status: string;
  projectId: string;
  serviceLineId?: string;
}

// — API Response wrapper —————
export interface ApiResponse<T> {
```

```
data?: T;
error?: string;
message?: string;
}

// — Dashboard KPI —————
export interface KpiData {
  totalAccounts: number;
  activeAccounts: number;
  totalOpportunities: number;
  highPriorityAccounts: number;
  serviceLineCount: number;
}

// — Filter state —————
export interface ProjectFilters {
  search: string;
  region: string;
  industry: string;
  status: string;
  priority: string;
}
```

Phase 5 — Database Design

Entity Relationship Diagram

```
USER
  id, name, email, passwordHash, role, createdAt
  └─ (creates/owns Projects)

PROJECT (Account)
  id, accountName, accountId, description,
  ibu, region, industry, zone,
  status, priority, tier, createdAt, updatedAt
  └─ SERVICE_LINE (one-to-many)
      id, name, description, projectId
  └─ OPPORTUNITY (one-to-many)
      id, name, description, stage,
      priority, estimatedValue, status,
      projectId, serviceLineId
```

Relationships Explained

Relationship	Type	Meaning
Project → ServiceLine	One-to-Many	One project can have many service lines
Project → Opportunity	One-to-Many	One project can have many opportunities
ServiceLine → Opportunity	One-to-Many (optional)	An opportunity can belong to a specific service line

Phase 6 — Prisma Setup & Schema

Initialize Prisma

```
npx prisma init
```

This creates `prisma/schema.prisma` and adds `DATABASE_URL` to your `.env` file.

File: `.env`

```
DATABASE_URL="postgresql://postgres:yourpassword@localhost:5432/mini_pinnacle"  
NEXTAUTH_SECRET="your-random-secret-string-change-this"  
NEXTAUTH_URL="http://localhost:3000"
```



Generate a real secret with: `openssl rand -base64 32`

File: `prisma/schema.prisma`

```
generator client {  
  provider = "prisma-client-js"  
}  
  
datasource db {  
  provider = "postgresql"  
  url      = env("DATABASE_URL")  
}  
  
enum Role {  
  SUPER_ADMIN  
  MANAGER  
  END_USER  
}  
  
enum Status {  
  ACTIVE  
  INACTIVE  
  PENDING  
}  
  
enum Priority {  
  HIGH  
  MEDIUM  
  LOW  
}  
  
model User {  
  id           String    @id @default(cuid())  
  name        String  
  email       String    @unique  
  passwordHash String  
  role        Role      @default(END_USER)  
  createdAt   DateTime  @default(now())  
  updatedAt   DateTime  @updatedAt  
}
```

```

model Project {
  id          String          @id @default(cuid())
  accountName String
  accountId   String          @unique
  description  String?
  ibu         String
  region      String
  industry    String
  zone        String
  status      Status          @default(ACTIVE)
  priority    Priority         @default(MEDIUM)
  tier         String
  createdAt   DateTime        @default(now())
  updatedAt   DateTime        @updatedAt
  serviceLines ServiceLine[]
  opportunities Opportunity[]
}

model ServiceLine {
  id          String          @id @default(cuid())
  name        String
  description  String?
  projectId   String
  project     Project         @relation(fields: [projectId], references: [id],
onDelete: Cascade)
  opportunities Opportunity[]
}

model Opportunity {
  id          String          @id @default(cuid())
  name        String
  description  String?
  stage       String
  priority    Priority         @default(MEDIUM)
  estimatedValue Float?
  status      String
  projectId   String
  project     Project         @relation(fields: [projectId], references: [id],
onDelete: Cascade)
  serviceLineId String?
  serviceLine ServiceLine? @relation(fields: [serviceLineId], references: [id])
}

```

Apply Schema to Database

```

npx prisma generate      # generates the Prisma Client (TypeScript types)
npx prisma migrate dev --name init  # creates tables in PostgreSQL
npx prisma studio        # opens a GUI to view/edit your data in browser

```

File: lib/prisma.ts

This file creates ONE shared Prisma client for the whole app (prevents too many database connections in development).

```

import { PrismaClient } from "@prisma/client";

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
}

```

```
};

export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient({
    log: ["query"],
  });

if (process.env.NODE_ENV !== "production") globalForPrisma.prisma = prisma;
```

Phase 7 — Authentication (NextAuth)

We use NextAuth (Auth.js) because it handles sessions, cookies, token rotation, and CSRF protection automatically. Building this from scratch would take weeks and introduce security holes.

File: lib/auth.ts

```
import NextAuth from "next-auth";
import Credentials from "next-auth/providers/credentials";
import { prisma } from "@lib/prisma";
import bcrypt from "bcryptjs";

export const { handlers, auth, signIn, signOut } = NextAuth({
  providers: [
    Credentials({
      name: "credentials",
      credentials: {
        email: { label: "Email", type: "email" },
        password: { label: "Password", type: "password" },
      },
      async authorize(credentials) {
        if (!credentials?.email || !credentials?.password) return null;

        const user = await prisma.user.findUnique({
          where: { email: credentials.email as string },
        });

        if (!user) return null;

        const isValid = await bcrypt.compare(
          credentials.password as string,
          user.passwordHash
        );

        if (!isValid) return null;

        return { id: user.id, name: user.name, email: user.email, role: user.role };
      },
    }),
  ],
  callbacks: {
    jwt({ token, user }) {
      if (user) {
        token.role = (user as any).role;
        token.id = user.id;
      }
      return token;
    },
    session({ session, token }) {
      if (session.user) {
        (session.user as any).role = token.role;
        (session.user as any).id = token.id;
      }
      return session;
    },
  },
  pages: {
    signIn: "/login",
  },
  session: { strategy: "jwt" },
});
```

File: app/api/auth/[...nextauth]/route.ts

```
import { handlers } from "@lib/auth";
export const { GET, POST } = handlers;
```

File: app/(auth)/login/page.tsx

```
"use client";
import { useState } from "react";
import { signIn } from "next-auth/react";
import { useRouter } from "next/navigation";

export default function LoginPage() {
  const router = useRouter();
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setLoading(true);
    setError("");

    const result = await signIn("credentials", {
      email,
      password,
      redirect: false,
    });

    if (result?.error) {
      setError("Invalid email or password");
    } else {
      router.push("/dashboard");
    }
    setLoading(false);
  }

  return (
    <div className="min-h-screen bg-slate-100 flex items-center justify-center">
      <div className="bg-white rounded-xl shadow-lg p-8 w-full max-w-md">
        <h1 className="text-2xl font-bold text-slate-800 mb-2">Mini Pinnacle</h1>
        <p className="text-slate-500 mb-6">Sign in to your account</p>

        {error && (
          <div className="bg-red-50 text-red-600 text-sm px-4 py-3 rounded-lg mb-4">
            {error}
          </div>
        )}

        <form onSubmit={handleSubmit} className="space-y-4">
          <div>
            <label className="block text-sm font-medium text-slate-700 mb-1">
              Email
            </label>
            <input
```

```

        type="email"
        value={email}
        onChange={ (e) => setEmail(e.target.value) }
        required
        className="w-full border border-slate-300 rounded-lg px-3 py-2 text-sm
focus:outline-none focus:ring-2 focus:ring-blue-500"
        placeholder="you@company.com"
      />
    </div>
    <div>
      <label className="block text-sm font-medium text-slate-700 mb-1">
        Password
      </label>
      <input
        type="password"
        value={password}
        onChange={ (e) => setPassword(e.target.value) }
        required
        className="w-full border border-slate-300 rounded-lg px-3 py-2 text-sm
focus:outline-none focus:ring-2 focus:ring-blue-500"
        placeholder="....."
      />
    </div>
    <button
      type="submit"
      disabled={loading}
      className="w-full bg-blue-600 hover:bg-blue-700 text-white font-medium
py-2 px-4 rounded-lg transition-colors disabled:opacity-50"
    >
      {loading ? "Signing in..." : "Sign In"}
    </button>
  </form>
</div>
</div>
);
}

```

File: middleware.ts (protect all routes)

```

export { auth as middleware } from "@lib/auth";

export const config = {
  matcher: ["/((?!api/auth|_next/static|_next/image|favicon.ico|login).*)"],
};

```

Seed Script: Create First Admin User

Create prisma/seed.ts to add your first user to the database:

```

import { PrismaClient } from "@prisma/client";
import bcrypt from "bcryptjs";

const prisma = new PrismaClient();

async function main() {
  const passwordHash = await bcrypt.hash("admin123", 12);

  await prisma.user.upsert({

```

```

    where: { email: "admin@minipinnacle.com" },
    update: {},
    create: {
      name: "Super Admin",
      email: "admin@minipinnacle.com",
      passwordHash,
      role: "SUPER_ADMIN",
    },
  });

  console.log("✅ Admin user created: admin@minipinnacle.com / admin123");
}

main().catch(console.error).finally(() => prisma.$disconnect());

```

Add to package.json scripts section:

```

"prisma": {
  "seed": "ts-node --compiler-options {\"module\": \"CommonJS\"} prisma/seed.ts"
}

```

```

npx prisma db seed

```

Phase 8 — Role-Based Authorization

Permission Matrix

Feature	SUPER_ADMIN	MANAGER	END_USER
View Dashboard	✓	✓	✓
View Projects	✓	✓	✓
Create Project	✓	✓	✗
Edit Project	✓	✓	✗
Delete Project	✓	✓	✗
Manage Users	✓	✗	✗
View Users	✓	✗	✗

File: lib/permissions.ts

```
import { Role } from "@types";

export function canCreateProject(role: Role): boolean {
  return role === "SUPER_ADMIN" || role === "MANAGER";
}

export function canEditProject(role: Role): boolean {
  return role === "SUPER_ADMIN" || role === "MANAGER";
}

export function canDeleteProject(role: Role): boolean {
  return role === "SUPER_ADMIN" || role === "MANAGER";
}

export function canManageUsers(role: Role): boolean {
  return role === "SUPER_ADMIN";
}
```

How to Use in API Routes

Always check the session AND the role inside every API route that modifies data:

```
import { auth } from "@lib/auth";
import { canCreateProject } from "@lib/permissions";

export async function POST(request: Request) {
  const session = await auth();
  if (!session) {
    return Response.json({ error: "Unauthorized" }, { status: 401 });
  }

  const role = (session.user as any).role;
  if (!canCreateProject(role)) {
    return Response.json({ error: "Forbidden" }, { status: 403 });
  }

  // ... proceed with creating project
}
```


Phase 9 — Dashboard

File: components/dashboard/KpiCard.tsx

```
interface KpiCardProps {
  title: string;
  value: number | string;
  description?: string;
  color?: "blue" | "green" | "orange" | "red";
}

const colorMap = {
  blue: "bg-blue-50 border-blue-200 text-blue-700",
  green: "bg-green-50 border-green-200 text-green-700",
  orange: "bg-orange-50 border-orange-200 text-orange-700",
  red: "bg-red-50 border-red-200 text-red-700",
};

export default function KpiCard({
  title, value, description, color = "blue"
}: KpiCardProps) {
  return (
    <div className={`border rounded-xl p-5 ${colorMap[color]}`>
      <p className="text-sm font-medium opacity-75">{title}</p>
      <p className="text-3xl font-bold mt-1">{value}</p>
      {description && (
        <p className="text-xs mt-1 opacity-60">{description}</p>
      )}
    </div>
  );
}
```

File: components/dashboard/FilterBar.tsx

```
"use client";

interface FilterBarProps {
  onFilterChange: (key: string, value: string) => void;
  filters: Record<string, string>;
}

export default function FilterBar({ onFilterChange, filters }: FilterBarProps) {
  const regions = ["All", "APAC", "EMEA", "AMER", "LATAM"];
  const industries = ["All", "Banking", "Healthcare", "Retail", "Technology", "Manufacturing"];
  const statuses = ["All", "ACTIVE", "INACTIVE", "PENDING"];

  return (
    <div className="flex flex-wrap gap-3 bg-white p-4 rounded-xl border border-slate-200">
      {[
        { label: "Region", key: "region", options: regions },
        { label: "Industry", key: "industry", options: industries },
        { label: "Status", key: "status", options: statuses },
      ].map(({ label, key, options }) => (
        <div key={key}>
          <label className="text-xs text-slate-500 block mb-1">{label}</label>
          <select
```

```

        value={filters[key] ?? "All"}
        onChange={(e) => onFilterChange(key, e.target.value)}
        className="text-sm border border-slate-300 rounded-lg px-3 py-1.5
focus:outline-none focus:ring-2 focus:ring-blue-500"
      >
        {options.map((o) => (
          <option key={o}>{o}</option>
        ))}
      </select>
    </div>
  ))}
</div>
);
}

```

File: app/api/dashboard/kpi/route.ts

```

import { auth } from "@lib/auth";
import { prisma } from "@lib/prisma";

export async function GET() {
  const session = await auth();
  if (!session) return Response.json({ error: "Unauthorized" }, { status: 401 });

  const [total, active, opportunities, highPriority, serviceLines] = await
Promise.all([
  prisma.project.count(),
  prisma.project.count({ where: { status: "ACTIVE" } }),
  prisma.opportunity.count(),
  prisma.project.count({ where: { priority: "HIGH" } }),
  prisma.serviceLine.count(),
]);

  return Response.json({
    totalAccounts: total,
    activeAccounts: active,
    totalOpportunities: opportunities,
    highPriorityAccounts: highPriority,
    serviceLineCount: serviceLines,
  });
}

```

File: app/dashboard/page.tsx

```

"use client";
import { useEffect, useState } from "react";
import KpiCard from "@components/dashboard/KpiCard";
import FilterBar from "@components/dashboard/FilterBar";
import type { KpiData } from "@types";

export default function DashboardPage() {
  const [kpi, setKpi] = useState<KpiData | null>(null);
  const [filters, setFilters] = useState<Record<string, string>>({});

  useEffect(() => {
    fetch("/api/dashboard/kpi")
      .then(r => r.json())
      .then(setKpi);
  });
}

```

```

}, []);

function handleFilterChange(key: string, value: string) {
  setFilters((prev) => ({ ...prev, [key]: value }));
}

return (
  <div className="space-y-6">
    <div>
      <h2 className="text-2xl font-bold text-slate-800">Dashboard</h2>
      <p className="text-slate-500">Executive overview of all accounts</p>
    </div>

    <FilterBar filters={filters} onFilterChange={handleFilterChange} />

    {kpi ? (
      <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4">
        <KpiCard title="Total Accounts" value={kpi.totalAccounts}
color="blue" />
        <KpiCard title="Active Accounts" value={kpi.activeAccounts}
color="green" />
        <KpiCard title="Opportunities" value={kpi.totalOpportunities}
color="orange" />
        <KpiCard title="High Priority" value={kpi.highPriorityAccounts}
color="red" />
        <KpiCard title="Service Lines" value={kpi.serviceLineCount}
color="blue" />
      </div>
    ) : (
      <p className="text-slate-400">Loading KPIs...</p>
    )}
  </div>
);
}

```

Phase 10 — Projects / Accounts Listing

File: [app/api/projects/route.ts](#)

```
import { auth } from "@lib/auth";
import { prisma } from "@lib/prisma";
import { canCreateProject } from "@lib/permissions";

export async function GET(request: Request) {
  const session = await auth();
  if (!session) return Response.json({ error: "Unauthorized" }, { status: 401 });

  const { searchParams } = new URL(request.url);
  const search = searchParams.get("search") ?? "";
  const region = searchParams.get("region") ?? "";
  const industry = searchParams.get("industry") ?? "";
  const status = searchParams.get("status") ?? "";
  const priority = searchParams.get("priority") ?? "";

  const projects = await prisma.project.findMany({
    where: {
      AND: [
        search ? { accountName: { contains: search, mode: "insensitive" } } : {},
        region && region !== "All" ? { region } : {},
        industry && industry !== "All" ? { industry } : {},
        status && status !== "All" ? { status: status as any } : {},
        priority && priority !== "All" ? { priority: priority as any } : {},
      ],
    },
    orderBy: { createdAt: "desc" },
  });

  return Response.json(projects);
}

export async function POST(request: Request) {
  const session = await auth();
  if (!session) return Response.json({ error: "Unauthorized" }, { status: 401 });

  const role = (session.user as any).role;
  if (!canCreateProject(role)) {
    return Response.json({ error: "Forbidden" }, { status: 403 });
  }

  const body = await request.json();

  const project = await prisma.project.create({
    data: {
      accountName: body.accountName,
      accountId: body.accountId,
      description: body.description,
      ibu: body.ibu,
      region: body.region,
      industry: body.industry,
      zone: body.zone,
      status: body.status ?? "ACTIVE",
      priority: body.priority ?? "MEDIUM",
      tier: body.tier,
    },
  });
}
```

```
    return Response.json(project, { status: 201 });
  }
}
```

File: app/api/projects/[id]/route.ts

```
import { auth } from "@lib/auth";
import { prisma } from "@lib/prisma";
import { canEditProject, canDeleteProject } from "@lib/permissions";

export async function GET(_req: Request, { params }: { params: { id: string } }) {
  const session = await auth();
  if (!session) return Response.json({ error: "Unauthorized" }, { status: 401 });

  const project = await prisma.project.findUnique({
    where: { id: params.id },
    include: { serviceLines: true, opportunities: true },
  });

  if (!project) return Response.json({ error: "Not found" }, { status: 404 });
  return Response.json(project);
}

export async function PATCH(request: Request, { params }: { params: { id: string } }) {
  const session = await auth();
  if (!session) return Response.json({ error: "Unauthorized" }, { status: 401 });

  const role = (session.user as any).role;
  if (!canEditProject(role)) return Response.json({ error: "Forbidden" }, { status: 403 });

  const body = await request.json();
  const updated = await prisma.project.update({
    where: { id: params.id },
    data: body,
  });
  return Response.json(updated);
}

export async function DELETE(_req: Request, { params }: { params: { id: string } }) {
  const session = await auth();
  if (!session) return Response.json({ error: "Unauthorized" }, { status: 401 });

  const role = (session.user as any).role;
  if (!canDeleteProject(role)) return Response.json({ error: "Forbidden" }, { status: 403 });

  await prisma.project.delete({ where: { id: params.id } });
  return Response.json({ message: "Deleted successfully" });
}
```

File: components/projects/ProjectTable.tsx

```
"use client";
import Link from "next/link";
```

```

import type { Project } from "@types";

interface Props { projects: Project[]; }

const statusColors: Record<string, string> = {
  ACTIVE: "bg-green-100 text-green-700",
  INACTIVE: "bg-slate-100 text-slate-600",
  PENDING: "bg-yellow-100 text-yellow-700",
};

const priorityColors: Record<string, string> = {
  HIGH: "bg-red-100 text-red-700",
  MEDIUM: "bg-orange-100 text-orange-700",
  LOW: "bg-blue-100 text-blue-700",
};

export default function ProjectTable({ projects }: Props) {
  if (projects.length === 0) {
    return (
      <div className="text-center py-16 text-slate-400">
        No projects found. Try adjusting your filters.
      </div>
    );
  }

  return (
    <div className="overflow-x-auto rounded-xl border border-slate-200">
      <table className="w-full text-sm">
        <thead className="bg-slate-50 text-slate-600 text-left">
          <tr>
            {["Account Name", "Account ID", "Region", "IBU", "Industry", "Status", "Priority", "Tier"].map(h => (
              <th key={h} className="px-4 py-3 font-medium">{h}</th>
            ))}
          </tr>
        </thead>
        <tbody className="divide-y divide-slate-100">
          {projects.map((p) => (
            <tr key={p.id} className="hover:bg-slate-50 transition-colors">
              <td className="px-4 py-3">
                <Link href={`\`/projects/${p.id}`}` className="text-blue-600 hover:underline font-medium">
                  {p.accountName}
                </Link>
              </td>
              <td className="px-4 py-3 text-slate-500">{p.accountId}</td>
              <td className="px-4 py-3">{p.region}</td>
              <td className="px-4 py-3">{p.ibu}</td>
              <td className="px-4 py-3">{p.industry}</td>
              <td className="px-4 py-3">
                <span className={`px-2 py-0.5 rounded-full text-xs font-medium ${statusColors[p.status]}`}>
                  {p.status}
                </span>
              </td>
              <td className="px-4 py-3">
                <span className={`px-2 py-0.5 rounded-full text-xs font-medium ${priorityColors[p.priority]}`}>
                  {p.priority}
                </span>
              </td>
              <td className="px-4 py-3 text-slate-500">{p.tier}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

```

```

        </table>
    </div>
);
}

```

File: app/projects/page.tsx

```

use client";
import { useEffect, useState, useCallback } from "react";
import Link from "next/link";
import ProjectTable from "@components/projects/ProjectTable";
import type { Project, ProjectFilters } from "@types";

export default function ProjectsPage() {
  const [projects, setProjects] = useState<Project[]>([]);
  const [loading, setLoading] = useState(true);
  const [filters, setFilters] = useState<ProjectFilters>({
    search: "", region: "", industry: "", status: "", priority: "",
  });

  const fetchProjects = useCallback(async () => {
    setLoading(true);
    const params = new URLSearchParams();
    Object.entries(filters).forEach(([k, v]) => { if (v) params.set(k, v); });
    const res = await fetch(`/api/projects?${params}`);
    const data = await res.json();
    setProjects(data);
    setLoading(false);
  }, [filters]);

  useEffect(() => { fetchProjects(); }, [fetchProjects]);

  return (
    <div className="space-y-6">
      <div className="flex items-center justify-between">
        <div>
          <h2 className="text-2xl font-bold text-slate-800">Projects / Accounts</h2>
          <p className="text-slate-500">{projects.length} accounts found</p>
        </div>
        <Link href="/projects/new"
          className="bg-blue-600 hover:bg-blue-700 text-white text-sm font-medium px-4 py-2 rounded-lg transition-colors">
          + New Project
        </Link>
      </div>

      {/* Search bar */}
      <input
        type="text"
        placeholder="Search by account name..."
        value={filters.search}
        onChange={(e) => setFilters(f => ({ ...f, search: e.target.value })))}
        className="w-full border border-slate-300 rounded-lg px-4 py-2 text-sm
focus:outline-none focus:ring-2 focus:ring-blue-500"
      />

      {/* Filters */}
      <div className="flex gap-3 flex-wrap">
        {[
          { key: "region", options: ["", "APAC", "EMEA", "AMER", "LATAM"] },

```

```

    { key: "industry", options:
["","Banking","Healthcare","Retail","Technology"] },
    { key: "status", options: ["","ACTIVE","INACTIVE","PENDING"] },
    { key: "priority", options: ["","HIGH","MEDIUM","LOW"] },
  ].map(({ key, options }) => (
    <select key={key}
      value={filters[key as keyof ProjectFilters]}
      onChange={(e) => setFilters(f => ({ ...f, [key]: e.target.value })))}
      className="border border-slate-300 rounded-lg px-3 py-1.5 text-sm
capitalize"
    >
      <option value="">All {key}</option>
      {options.filter(Boolean).map(o => <option key={o}>{o}</option>)}
    </select>
  ))}
</div>

{loading ? (
  <p className="text-slate-400 text-center py-10">Loading...</p>
) : (
  <ProjectTable projects={projects} />
)}
</div>
);
}

```

Phase 11 — Create Project Form

File: [app/projects/new/page.tsx](#)

```
"use client";
import { useState } from "react";
import { useRouter } from "next/navigation";

export default function NewProjectPage() {
  const router = useRouter();
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const [form, setForm] = useState({
    accountName: "", accountId: "", description: "",
    ibu: "", region: "", industry: "", zone: "",
    status: "ACTIVE", priority: "MEDIUM", tier: "",
  });

  function handleChange(e: React.ChangeEvent<HTMLInputElement | HTMLSelectElement |
HTMLTextAreaElement>) {
    setForm(f => ({ ...f, [e.target.name]: e.target.value }));
  }

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setLoading(true);
    setError("");

    const res = await fetch("/api/projects", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(form),
    });

    if (res.ok) {
      router.push("/projects");
    } else {
      const data = await res.json();
      setError(data.error ?? "Failed to create project");
    }
    setLoading(false);
  }

  return (
    <div className="max-w-2xl">
      <h2 className="text-2xl font-bold text-slate-800 mb-6">Create New Project</h2>

      {error && (
        <div className="bg-red-50 text-red-600 px-4 py-3 rounded-lg mb-4 text-sm">{error}</div>
      )}

      <form onSubmit={handleSubmit} className="bg-white rounded-xl border border-slate-200 p-6 space-y-4">
        {[
          { label: "Account Name", name: "accountName", required: true },
          { label: "Account ID", name: "accountId", required: true },
          { label: "IBU", name: "ibu", required: true },
          { label: "Zone", name: "zone", required: true },
          { label: "Tier", name: "tier", required: true },
        ].map(({ label, name, required }) => (
```

```

        <div key={name}>
          <label className="block text-sm font-medium text-slate-700 mb-1">{label}</label>
          <input
            name={name}
            value={form[name as keyof typeof form]}
            onChange={handleChange}
            required={required}
            className="w-full border border-slate-300 rounded-lg px-3 py-2 text-sm focus:outline-none focus:ring-2 focus:ring-blue-500"
          />
        </div>
      )}

      {/* Dropdowns */}
      {[
        { label: "Region", name: "region", options: ["APAC", "EMEA", "AMER", "LATAM"] },
        { label: "Industry", name: "industry", options: ["Banking", "Healthcare", "Retail", "Technology", "Manufacturing"] },
        { label: "Status", name: "status", options: ["ACTIVE", "INACTIVE", "PENDING"] },
        { label: "Priority", name: "priority", options: ["HIGH", "MEDIUM", "LOW"] },
      ].map(({ label, name, options }) => (
        <div key={name}>
          <label className="block text-sm font-medium text-slate-700 mb-1">{label}</label>
          <select
            name={name}
            value={form[name as keyof typeof form]}
            onChange={handleChange}
            className="w-full border border-slate-300 rounded-lg px-3 py-2 text-sm focus:outline-none focus:ring-2 focus:ring-blue-500"
          >
            {options.map(o => <option key={o}>{o}</option>)}
          </select>
        </div>
      ))}

      <div>
        <label className="block text-sm font-medium text-slate-700 mb-1">Description</label>
        <textarea
          name="description"
          value={form.description}
          onChange={handleChange}
          rows={3}
          className="w-full border border-slate-300 rounded-lg px-3 py-2 text-sm focus:outline-none focus:ring-2 focus:ring-blue-500"
        />
      </div>

      <div className="flex gap-3 pt-2">
        <button type="submit" disabled={loading}
          className="bg-blue-600 hover:bg-blue-700 text-white text-sm font-medium px-6 py-2 rounded-lg disabled:opacity-50"
        >
          {loading ? "Creating..." : "Create Project"}
        </button>
        <button type="button" onClick={() => router.back()}
          className="border border-slate-300 text-slate-600 text-sm font-medium px-6 py-2 rounded-lg hover:bg-slate-50"
        >
          Cancel
        </button>
      </div>

```

```
    </form>
  </div>
);
}
```

Phase 12 — Project Detail Page & Tabs

File: `app/projects/[id]/page.tsx`

```
"use client";
import { useEffect, useState } from "react";
import { useParams } from "next/navigation";
import type { Project } from "@types";

type Tab = "overview" | "serviceLines" | "opportunities" | "accountDetails";

export default function ProjectDetailPage() {
  const params = useParams();
  const [project, setProject] = useState<Project | null>(null);
  const [tab, setTab] = useState<Tab>("overview");

  useEffect(() => {
    fetch(`/api/projects/${params.id}`)
      .then(r => r.json())
      .then(setProject);
  }, [params.id]);

  if (!project) return <p className="text-slate-400">Loading...</p>;

  const tabs: { key: Tab; label: string }[] = [
    { key: "overview", label: "Overview" },
    { key: "serviceLines", label: "Service Lines" },
    { key: "opportunities", label: "Opportunities" },
    { key: "accountDetails", label: "Account Details" },
  ];

  return (
    <div className="space-y-6">
      { /* Header */ }
      <div className="bg-white rounded-xl border border-slate-200 p-6">
        <div className="flex items-start justify-between">
          <div>
            <h2 className="text-2xl font-bold text-slate-800">{project.accountName}</h2>
            <p className="text-slate-500 text-sm mt-1">ID: {project.accountId}</p>
          </div>
          <div className="flex gap-2">
            <span className="bg-green-100 text-green-700 text-xs font-medium px-3 py-1 rounded-full">
              {project.status}
            </span>
            <span className="bg-red-100 text-red-700 text-xs font-medium px-3 py-1 rounded-full">
              {project.priority}
            </span>
          </div>
        </div>
        <div className="grid grid-cols-2 md:grid-cols-4 gap-4 mt-4 text-sm text-slate-600">
          <div><span className="font-medium">Region:</span> {project.region}</div>
          <div><span className="font-medium">Industry:</span> {project.industry}</div>
          <div><span className="font-medium">IBU:</span> {project.ibu}</div>
          <div><span className="font-medium">Tier:</span> {project.tier}</div>
        </div>
      </div>
    </div>
  );
}
```

```

    { /* Tabs */
    <div className="border-b border-slate-200">
      <nav className="flex gap-6">
        {tabs.map(t => (
          <button key={t.key} onClick={() => setTab(t.key)}
            className={`py-3 text-sm font-medium border-b-2 transition-colors ${
              tab === t.key
                ? "border-blue-600 text-blue-600"
                : "border-transparent text-slate-500 hover:text-slate-700"
            }`}
          >
            {t.label}
          </button>
        ))}
      </nav>
    </div>

    { /* Tab Content */
    <div className="bg-white rounded-xl border border-slate-200 p-6">
      {tab === "overview" && (
        <div>
          <h3 className="font-semibold text-slate-800 mb-2">Description</h3>
          <p className="text-slate-600">{project.description ?? "No description
provided."}</p>
        </div>
      )}
      {tab === "serviceLines" && (
        <div>
          <h3 className="font-semibold text-slate-800 mb-3">Service Lines</h3>
          {project.servicelines?.length ? (
            <ul className="space-y-2">
              {project.servicelines.map(sl => (
                <li key={sl.id} className="flex items-center gap-2 text-sm text-
slate-700">
                  <span className="w-2 h-2 bg-blue-500 rounded-full" />
                  {sl.name}
                </li>
              ))}
            </ul>
          ) : (
            <p className="text-slate-400">No service lines added yet.</p>
          )}
        </div>
      )}
      {tab === "opportunities" && (
        <div>
          <h3 className="font-semibold text-slate-800 mb-3">Opportunities</h3>
          {project.opportunities?.length ? (
            <table className="w-full text-sm">
              <thead><tr className="text-left text-slate-500">
                <th className="pb-2">Name</th>
                <th className="pb-2">Stage</th>
                <th className="pb-2">Priority</th>
                <th className="pb-2">Value</th>
              </tr></thead>
              <tbody>
                {project.opportunities.map(o => (
                  <tr key={o.id} className="border-t border-slate-100">
                    <td className="py-2">{o.name}</td>
                    <td>{o.stage}</td>
                    <td>{o.priority}</td>
                    <td>{o.estimatedValue ? `${o.estimatedValue.toLocaleString()}`
: "-"}</td>
                  </tr>
                ))}
              </tbody>
            </table>
          )}
        </div>
      )}
    </div>
  )}

```

```

        </tbody>
      </table>
    ) : (
      <p className="text-slate-400">No opportunities yet.</p>
    )
  </div>
)}
{tab === "accountDetails" && (
  <div className="grid grid-cols-2 gap-4 text-sm">
    {[
      ["Account Name", project.accountName],
      ["Account ID", project.accountId],
      ["Region", project.region],
      ["Industry", project.industry],
      ["IBU", project.ibu],
      ["Zone", project.zone],
      ["Tier", project.tier],
      ["Status", project.status],
      ["Priority", project.priority],
      ["Created", new Date(project.createdAt).toLocaleDateString()],
    ].map(([label, value]) => (
      <div key={label}>
        <span className="font-medium text-slate-700">{label}: </span>
        <span className="text-slate-600">{value}</span>
      </div>
    ))}
  </div>
)}
</div>
</div>
);
}

```

Phase 13 — Service Lines Module

File: [app/api/service-lines/route.ts](#)

```
import { auth } from "@lib/auth";
import { prisma } from "@lib/prisma";
import { canEditProject } from "@lib/permissions";

export async function POST(request: Request) {
  const session = await auth();
  if (!session) return Response.json({ error: "Unauthorized" }, { status: 401 });

  const role = (session.user as any).role;
  if (!canEditProject(role)) return Response.json({ error: "Forbidden" }, { status: 403 });

  const body = await request.json();
  const sl = await prisma.serviceLine.create({
    data: {
      name: body.name,
      description: body.description,
      projectId: body.projectId,
    },
  });
  return Response.json(sl, { status: 201 });
}
```

File: [app/api/service-lines/\[id\]/route.ts](#)

```
import { auth } from "@lib/auth";
import { prisma } from "@lib/prisma";
import { canEditProject } from "@lib/permissions";

export async function DELETE(_req: Request, { params }: { params: { id: string } }) {
  const session = await auth();
  if (!session) return Response.json({ error: "Unauthorized" }, { status: 401 });

  const role = (session.user as any).role;
  if (!canEditProject(role)) return Response.json({ error: "Forbidden" }, { status: 403 });

  await prisma.serviceLine.delete({ where: { id: params.id } });
  return Response.json({ message: "Deleted" });
}
```

Phase 15 — Users Module (Super Admin Only)

File: [app/api/users/route.ts](#)

```
import { auth } from "@lib/auth";
import { prisma } from "@lib/prisma";
import { canManageUsers } from "@lib/permissions";
import bcrypt from "bcryptjs";

export async function GET() {
  const session = await auth();
  if (!session) return Response.json({ error: "Unauthorized" }, { status: 401 });

  const role = (session.user as any).role;
  if (!canManageUsers(role)) return Response.json({ error: "Forbidden" }, { status: 403 });

  const users = await prisma.user.findMany({
    select: { id: true, name: true, email: true, role: true, createdAt: true },
    orderBy: { createdAt: "desc" },
  });
  return Response.json(users);
}

export async function POST(request: Request) {
  const session = await auth();
  if (!session) return Response.json({ error: "Unauthorized" }, { status: 401 });

  const role = (session.user as any).role;
  if (!canManageUsers(role)) return Response.json({ error: "Forbidden" }, { status: 403 });

  const body = await request.json();
  const passwordHash = await bcrypt.hash(body.password, 12);

  try {
    const user = await prisma.user.create({
      data: {
        name: body.name,
        email: body.email,
        passwordHash,
        role: body.role ?? "END_USER",
      },
      select: { id: true, name: true, email: true, role: true },
    });
    return Response.json(user, { status: 201 });
  } catch {
    return Response.json({ error: "Email already exists" }, { status: 400 });
  }
}
```

File: [app/users/page.tsx](#)

```
"use client";
import { useEffect, useState } from "react";
import type { User } from "@types";
```

```

export default function UsersPage() {
  const [users, setUsers] = useState<User[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
    fetch("/api/users")
      .then(async r => {
        if (!r.ok) { setError("Access denied"); return; }
        setUsers(await r.json());
      })
      .finally(() => setLoading(false));
  }, []);

  if (loading) return <p className="text-slate-400">Loading...</p>;
  if (error) return <p className="text-red-500">{error}</p>;

  return (
    <div className="space-y-6">
      <h2 className="text-2xl font-bold text-slate-800">Users</h2>
      <div className="bg-white rounded-xl border border-slate-200 overflow-hidden">
        <table className="w-full text-sm">
          <thead className="bg-slate-50 text-slate-600">
            <tr>
              {["Name", "Email", "Role", "Joined"].map(h => (
                <th key={h} className="text-left px-4 py-3 font-medium">{h}</th>
              ))}
            </tr>
          </thead>
          <tbody className="divide-y divide-slate-100">
            {users.map(u => (
              <tr key={u.id} className="hover:bg-slate-50">
                <td className="px-4 py-3 font-medium text-slate-800">{u.name}</td>
                <td className="px-4 py-3 text-slate-500">{u.email}</td>
                <td className="px-4 py-3">
                  <span className="bg-blue-100 text-blue-700 text-xs font-medium px-2
py-0.5 rounded-full">
                    {u.role}
                  </span>
                </td>
                <td className="px-4 py-3 text-slate-500">
                  {new Date(u.createdAt).toLocaleDateString()}
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  );
}

```

Phase 16 — Profile Page

File: `app/profile/page.tsx`

```
"use client";
import { useSession } from "next-auth/react";

export default function ProfilePage() {
  const { data: session } = useSession();
  const user = session?.user as any;

  if (!user) return <p className="text-slate-400">Loading...</p>;

  return (
    <div className="max-w-lg space-y-6">
      <h2 className="text-2xl font-bold text-slate-800">My Profile</h2>
      <div className="bg-white rounded-xl border border-slate-200 p-6 space-y-4">
        <div className="w-16 h-16 rounded-full bg-blue-100 flex items-center justify-center text-2xl font-bold text-blue-600">
          {user.name?.charAt(0)}
        </div>
        {[
          { label: "Name", value: user.name },
          { label: "Email", value: user.email },
          { label: "Role", value: user.role },
        ].map(({ label, value }) => (
          <div key={label}>
            <p className="text-xs text-slate-500 font-medium uppercase tracking-wide">{label}</p>
            <p className="text-slate-800 mt-0.5">{value}</p>
          </div>
        ))}
      </div>
    </div>
  );
}
```

Phase 19 — Custom Hooks

File: `hooks/useDebounce.ts`

Prevents API call on every single keystroke — waits until user stops typing for 300ms.

```
import { useEffect, useState } from "react";

export function useDebounce<T>(value: T, delay = 300): T {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);

  return debounced;
}
```

File: `hooks/useProjects.ts`

```
import { useEffect, useState } from "react";
import type { Project, ProjectFilters } from "@types";

export function useProjects(filters: ProjectFilters) {
  const [projects, setProjects] = useState<Project[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
    const params = new URLSearchParams();
    Object.entries(filters).forEach(([k, v]) => { if (v) params.set(k, v); });

    setLoading(true);
    fetch(`/api/projects?${params}`)
      .then(r => r.json())
      .then(setProjects)
      .catch(() => setError("Failed to load projects"))
      .finally(() => setLoading(false));
  }, [filters]);

  return { projects, loading, error };
}
```

Phase 22 — Git Workflow

Commit after every meaningful milestone. Here is the recommended commit sequence:

Milestone	Commit Message
Initial setup	chore: initial Next.js project setup
Folder structure	chore: create project folder structure
Layout components	feat: add Sidebar and Header layout
TypeScript types	feat: add TypeScript type definitions
Prisma schema	feat: add Prisma schema and database models
Authentication	feat: add NextAuth credential authentication
Middleware	feat: add route protection middleware
Dashboard	feat: add dashboard with KPI cards
Projects listing	feat: add projects listing with search and filters
Create project	feat: add create project form
Project detail	feat: add project detail page with tabs
Users module	feat: add users module for Super Admin
Profile page	feat: add profile page
Custom hooks	feat: add useDebounce and useProjects hooks

```
git add .
git commit -m "feat: add dashboard with KPI cards"
git push
```

Phase 23 — Manual Testing Checklist

Test	Steps	Expected
Login — valid	Go to /login. Enter admin@minipinnacle.com / admin123. Click Sign In.	Redirected to /dashboard
Login — invalid	Enter wrong password.	Error message shown
Protected route	Log out. Visit /dashboard in browser.	Redirected to /login
Dashboard KPIs	Log in. Go to /dashboard.	KPI cards show numbers
Projects list	Go to /projects.	Table of accounts renders
Search	Type in search box.	Table filters as you type
Filter by region	Select 'APAC' in Region dropdown.	Only APAC accounts show
Create project — MANAGER	Log in as Manager. Click New Project. Fill form.	Project created, redirected to list
Create project — END_USER	Log in as End User. Try POST /api/projects.	403 Forbidden returned
Project detail	Click account name in table.	Detail page opens with header and tabs
Tab navigation	Click each tab (Overview, Service Lines...).	Tab content changes
Users page — SUPER_ADMIN	Log in as Super Admin. Go to /users.	User list renders
Users page — MANAGER	Log in as Manager. Go to /users.	Access denied message shown
Profile	Go to /profile.	Name, email, role displayed correctly
Logout	Click Logout in header.	Redirected to /login

Phase 24 — Final Project Tree

```
mini-pinnacle/
├── app/
│   ├── (auth)/login/page.tsx      ← Login page
│   ├── dashboard/page.tsx        ← Dashboard with KPIs
│   ├── projects/
│   │   ├── page.tsx              ← Project listing
│   │   ├── new/page.tsx          ← Create project form
│   │   └── [id]/page.tsx         ← Project detail + tabs
│   ├── users/page.tsx            ← Users list (Admin only)
│   ├── profile/page.tsx          ← Profile page
│   └── api/
│       ├── auth/[...nextauth]/route.ts ← NextAuth handler
│       ├── dashboard/kpi/route.ts  ← KPI endpoint
│       ├── projects/
│       │   ├── route.ts           ← GET list + POST create
│       │   └── [id]/route.ts      ← GET one + PATCH + DELETE
│       ├── service-lines/
│       │   ├── route.ts           ← POST create
│       │   └── [id]/route.ts      ← DELETE
│       └── users/route.ts         ← GET list + POST create
│   ├── layout.tsx                ← Root layout (Sidebar + Header)
│   └── globals.css
├── components/
│   ├── layout/
│   │   ├── Sidebar.tsx
│   │   └── Header.tsx
│   ├── dashboard/
│   │   ├── KpiCard.tsx
│   │   └── FilterBar.tsx
│   └── projects/
│       └── ProjectTable.tsx
├── hooks/
│   ├── useProjects.ts
│   └── useDebounce.ts
├── lib/
│   ├── prisma.ts                 ← Shared Prisma client
│   ├── auth.ts                  ← NextAuth configuration
│   └── permissions.ts            ← Role permission helpers
├── types/
│   └── index.ts                  ← All TypeScript interfaces
├── prisma/
│   ├── schema.prisma             ← Database schema
│   └── seed.ts                   ← Seed first admin user
├── middleware.ts                 ← Protect all routes
├── .env                          ← Secrets (never commit this)
└── package.json
```

Quick Reference — Where to Change Things

If I want to change...	Go to...
Dashboard UI	app/dashboard/page.tsx
KPI card design	components/dashboard/KpiCard.tsx
Sidebar links	components/layout/Sidebar.tsx
Header / logout	components/layout/Header.tsx
Project table columns	components/projects/ProjectTable.tsx
Project detail tabs	app/projects/[id]/page.tsx
Create project form fields	app/projects/new/page.tsx
Projects API (GET/POST)	app/api/projects/route.ts

If I want to change...	Go to...
Single project API (PATCH/DELETE)	app/api/projects/[id]/route.ts
Database schema	prisma/schema.prisma (then run migration)
Auth / login logic	lib/auth.ts
Who can do what	lib/permissions.ts
TypeScript types	types/index.ts
Login page	app/(auth)/login/page.tsx
Which routes are protected	middleware.ts